Programlama Dilleri

Dr. Öğr. Üyesi Hayri Volkan Agun

Bilgisayar Mühendisliği Bölümü

Bursa Teknik Üniversitesi

Statik Ata (Anchestor)

- Bir alt programın (subprogram) statik (lexical) kapsamı içinde, doğrudan veya dolaylı olarak iç içe bulunduğu tüm üst programlardır.
- Yani, programın kaynak kodundaki yapısına göre belirlenir.
- Değişken erişimi ve referans ortamını (referencing environment) belirler.

```
def main():
    x = 10  # static ancestor için değişken

    def sub1():
        y = 20

        def sub2():
            z = 30
            print(x)  # x burada statik ancestor
```

Dinamik Ata (Anchestor)

- Bir alt programın çalışması sırasında, o anda çağrı zincirinde (call stack) bulunan tüm üst programlardır.
- Yani, programın çalışma zamanı çağrı sırasına göre belirlenir.
- sub2 çalışırken dynamic ancestors:
  - sub1, main
  - Çünkü şu anda çağrı yığını (call stack) içinde bunlar mevcut

```
def main():
    def sub1():
        de
            print("hello")
        sub2()
    sub1()

main()
```

Ask ChatGPT

Soru

```
var x; // global

function sub1() {
    var x;  // sub1 scope
    function sub2() {
        // ...
    }
}

function sub3() {
    // ...
}
```

Ve çağrı sırası: main → sub1 → sub2 → sub3.

- Statik kapsam varsayıldığında, aşağıdaki durumlarda x referansı için doğru x deklarasyonu hangisidir?
    i. sub1
    ii. sub2
    iii. sub3
- a kısmını tekrar edin, ancak bu sefer dinamik kapsam varsayın.

Soru - Statik Skop

Fonksiyon	Referans edilen `x` hangi deklarasyonu kullanır?	Açıklama
`sub1`	`sub1` içindeki `var x`	`sub1` içinde kendi `x` var, en yakın scope bu
`sub2`	`sub1` içindeki `var x`	`sub2` içinde `x` yok, statik olarak **en yakın enclosing scope = sub1**
`sub3`	Global `var x`	`sub3` global scope'ta tanımlı, kendi `x` yok, statik ancestor = global

Soru – Dinamik Skop

Fonksiyon	Dynamic `x`	Açıklama
`sub1`	sub1.x	`sub1` çağrılıyor, kendi x'i var → kullanılır
`sub2`	sub1.x	`sub2` çağrılıyor, call stack: sub2 → sub1 → main → global; **sub2 içinde yok, sub1'de var → kullanılır**
`sub3`	sub2'de yok → sub1'de var? → evet → sub1.x	`sub3` çağrısı sub2'den → call stack: sub3 → sub2 → sub1 → main → global; en yakın `x` = **sub1.x**

Statik Değişkenler

- Bellekte programın tüm çalışması boyunca sabit bir yer kaplar.
- Derleme zamanında (compile-time) boyutu ve adresi belirlenir.
- Örnek: static int x; (C/C++), final sınıf değişkenleri (Java)
- Avantajlar:
  - Hızlı erişim (adres derleme zamanında bilinir)
  - Tekrar kullanım kolay, ömür programın sonuna kadar
- Dezavantajlar:
  - Esnek değil, boyutu çalışma zamanında değiştirilemez
  - Bellek israfı olabilir (hiç kullanılmasa bile yer kaplar)

Stack-Dynamic Değişkenler

- Fonksiyon çağrısı sırasında bellekte ayrılır (stack frame içinde)
  - Fonksiyon bitince otomatik olarak silinir
  - Örnek: Yerel değişkenler (int x; bir fonksiyon içinde C/C++)
- Avantajlar:
  - Otomatik yönetim : bellek sızıntısı riski yok
  - Fonksiyon çağrısına özel, tekrar kullanılabilir
- Dezavantajlar:
  - Ömrü kısadır, fonksiyon bitince yok olur
  - Çok büyük veri yapıları stack overflow riski taşır

Explicit Heap-Dynamic Değişkenler

- Programcı tarafından dinamik olarak heap üzerinde ayrılır ve serbest bırakılır
- Örnekler:
  - C/C++: malloc(), free()
  - C++: new, delete
- Avantajlar:
  - Ömür ve boyut çalışma zamanında belirlenebilir
  - Büyük veri yapıları için uygundur
  - Esnek kullanım (farklı veri tiplerini aynı blokta depolayabilir)
- Dezavantajlar:
  - Programcı yönetmek zorundadır, memory leak riski
  - Daha yavaş erişim, heap üzerinde adres hesaplaması gerektirir.

Referencing Environment

- Bir ifadenin (statement) referencing environment'ı:
  - O ifadede erişilebilir tüm değişkenlerin kümesidir.
  - Başka bir deyişle: Bir satır kodun "hangi değişkenleri görebildiği"
- Statik skop için bir ifadenin erişebileceği değişkenler:
  - Yerel değişkenler (local scope)
  - Tüm üst (ancestor) scope'lardaki değişkenler.
  - Yakın scope'taki tanımlar, uzak olanları gizler (shadowing)

```
def sub1():
    a = 5;  # Creates a local
    b = 7;  # Creates another local
    ...  <------------------------------------ 1
  def sub2():
    global g;  # Global g is now assignable here

    c = 9;  # Creates a new local
    ...  <------------------------------------ 2
    def sub3():
      nonlocal c:  # Makes nonlocal c visible here
      g = 11;  # Creates a new local
      ...  <------------------------------------ 3
```

1. Yerel a ve b (sub1'e ait), referans için global g ancak atama için değil
2. Yerel c (sub2'ye ait), hem referans hem de atama için global g
3. Nonlocal c (sub2'ye ait, en yakın local olmayan skop) ama burada değiştirmek istiyorum, yerel g (sub3'e ait)

Referencing Environment

- Bir alt program (**subprogram**), yürütülmesi başlamış ancak henüz sona ermemişse **aktif** kabul edilir.
- Dinamik kapsamlı (dynamically scoped) bir dilde bir ifadenin referans ortamı (**referencing environment**), o ifadede yerel olarak tanımlanmış değişkenler ile o anda aktif olan diğer tüm alt programların değişkenlerinin birleşiminden oluşur.
- Yine, aktif alt programlardaki bazı değişkenler referans ortamından gizlenebilir. Daha yeni alt program çağrıları (aktivasyonları), önceki alt program çağrılarındaki aynı isimli değişkenleri gizleyen tanımlar içerebilir.

```
void sub1() {
  int a, b;
  ...  <------------------ 1
}  /* end of sub1 */
void sub2() {
  int b, c;
  ....  <------------------ 2
  sub1();
}  /* end of sub2 */
void main() {

  int c, d;
  ...  <------------------ 3
  sub2();
}  /* end of main */
```

1. sub1'in a ve b değişkenleri, sub2'nin c değişkeni, main'in d değişkeni (main'deki c ve sub2'deki b gizlenmiştir)
2. sub2'nin b ve c değişkenleri, main'in d değişkeni (main'deki c gizlenmiştir)
3. main'in c ve d değişkenleri)

Sabitler

- Okunabilirliği artırmak ve hata oranını azaltmak için bazı değişkenler değiştirilemez sabitler olarak tanımlanabilir.
- Örneğin yanda Java dilinde len final terimi ile bir sabit olarak tanımlanmıştır.
- Örneğin C++ da const terimi ile result bir sabit olarak tanımlanmaktadır.
- Bu ifadeler dışında sabitler enum tipinde de tanımlanabilir. Bu tür sabitleri önümüzdeki derslerde göreceğiz.

```
void example() {
  final int len = 100;
  int[] intList = new int[len];
  String[] strList = new String[len];
  ...
  for (index = 0; index < len; index++) {
    ...
  }
  ...
  for (index = 0; index < len; index++) {
    ...
  }
  ...
  average = sum / len;
  ...
}
```

```
const int result = 2 * width + 1;
```

Veri tipleri – İlkel tipler (Primitive)

- Primitive veri tipleri: Sayısal ve Mantıksal Tiplerdir.
- Kendi başına tanımlanan, başka türlere dayanmayan veri tipleridir.
- Çoğu dil temel set sağlar:
  - Bazıları donanım odaklı (integer),
  - bazıları yazılım destekli (long integer)
- Integer (tamsayı): signed/unsigned, çeşitli boyutlar (byte, short, int, long)
  - Python/F#: long integer → sınırsız uzunluk
  - Negatif tamsayı: iki tamamlayıcı (two's complement) yaygın
- Floating-point (kayan nokta): reel sayıların yaklaşık gösterimiI
  - EEE 754 standardı: float (4 byte),
  - double (8 byte)Precision ve range ile tanımlanır
- Complex: (a + bj) → çift floating-point
- Decimal: iş dünyası uygulamaları, tam ondalık hassasiyet
  - BCD (Binary Coded Decimal) kullanılır

Veri tipleri – İlkel tipler (Primitive)

- Boolean tipler: İki değer: true / false
  - Bayraklar, anahtarlar için okunabilir kullanım
  - Genellikle 1 byte üzerinde saklanır
- Karakter tipleri
  - Bilgisayarda sayısal kodlarla saklanır
  - ASCII (8-bit): 128 karakter
  - ISO 8859-1: 256 karakter
  - Unicode (UCS-2, UCS-4 / UTF-32): dünya dillerini kapsar
  - Java, JavaScript, Python, C#, F# Unicode destekler

Integer	Tamsayı, signed/unsigned	int, long
Floating-point	Reel sayı	float, double
Complex	Karmaşık sayı	7 + 3j
Decimal	Hassas ondalık sayı	decimal (C#, F#)
Boolean	True/False	bool
Character	Tek karakter veya string	char, Unicode string

Veri tipleri - Array

- Array aynı tipte veri elemanlarının oluşturduğu koleksiyondur.
- Her eleman indeks (subscript) ile erişilir
  - $array[index] \rightarrow element$
- Homogeneous (tüm elemanlar aynı tip)
- Bellekte genellikle ardışık (contiguous) saklanır
  - Hızlı erişim: O(1)
- Örnek
  - int A[5] = {1,2,3,4,5};

- İndeksleme (Subscript)
  - Dizi elemanına erişim: A[i]
  - İndeks:
    - Sabit → compile-time
    - Değişken → run-time hesaplama gerekir

$$Address = Base + (index \times element_size)$$

  - C/C++/Java → indeksler genelde integer
  - Ada → Boolean, char, enum da indeks olabilir

Array Türleri (Binding ve Allocation)

- **Static Array**
  - Boyut ve bellek compile-time
  - ✓ Çok hızlı
  - ❌ Esnek değil
- **Fixed Stack-Dynamic**
  - Stack’te oluşturulur
  - Boyut sabit, runtime’da belirlenir
  - ✓ Bellek verimli
- **Stack-Dynamic**
  - Boyut runtime’da belirlenir
  - ✓ Daha esnek
  - ❌ Allocation maliyeti var
- **Fixed Heap-Dynamic**
  - Heap’te oluşturulur (new, malloc)
  - Boyut sabit kalır.
- **Heap-Dynamic**
  - Boyut değişebilir (grow/shrink)
  - ✓ Maksimum esneklik
  - ❌ Yavaş, fragmentasyon riski

Heap vs Stack

Özellik	Stack	Heap
Yönetim	Otomatik	Manuel / Garbage Collector
Hız	Çok hızlı	Daha yavaş
Boyut	Küçük	Büyük
Ömür	Fonksiyon süresi	Program kontrolünde
Bellek düzeni	Düzenli	Dağınık olabilir
Hata türü	Stack overflow	Memory leak

Record

- Farklı türde veri elemanlarının bir araya geldiği yapı dır.
- Elemanlar **isimleriyle (field)** erişilir.
- Bellekte **ardışık (contiguous)** saklanır.
- **Özellikler:**
  - Heterojen veri yapısı (farklı tipler)
  - Örnek:
    - isim → string
    - numara → int
    - ortalama → float
- **Array ile farkı:**
  - Array → aynı tip
  - Record → farklı tipler
- **Örnek :**
  - Record içinde başka record olabilir (nested)
- **Alan erişimi:**
  - Dot notation:
    - employee.name
  - COBOL:
    - FIELD OF RECORD
- **Avantajlar:**
  - Okunabilirlik ↑ (isimli alanlar)
  - Tip güvenliği ↑
  - Heterojen veri modelleme kolay

Record

- **Array ile karşılaştırma:**
- Array:
  - Aynı veri tipi
  - Döngü ile işlenmesi kolay
- Record:
  - Farklı veri tipleri
  - Alan bazlı erişim
- **Performans:**
  - Field erişimi hızlı (offset ile)
- **Farklı dillerde:**
  - C / C++ / C# → struct
  - Java → class (data class)
  - Python / Ruby → dictionary (hash)

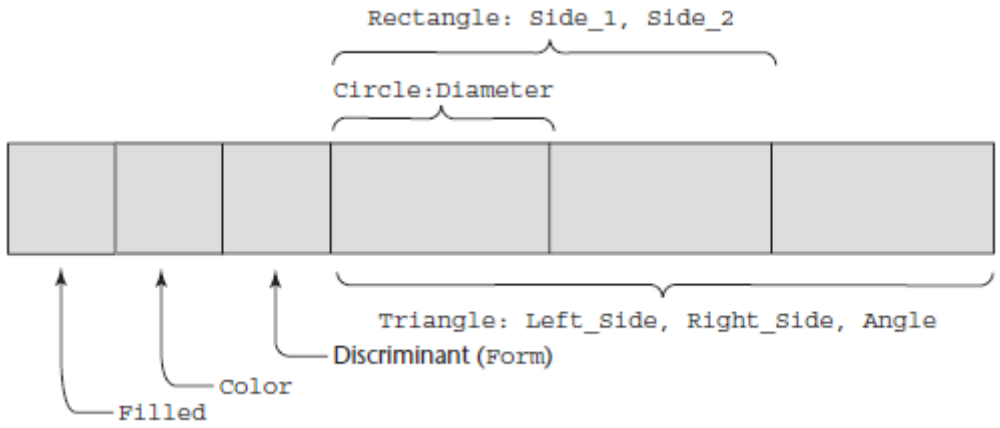
Tuple ve List

- **Tuple**
  - Record’a benzer ama:
  - Alan isimleri yok
- Genelde **immutable**
- **Örnek:**
  - (3, 5.8, "apple")
- **Kullanım:**
  - Fonksiyonlardan çoklu değer döndürme

- **List**
  - Dinamik veri yapısı
  - İlk olarak LISP’te kullanıldı
- **Özellikler:**
  - Python:
    - mutable
    - heterojen olabilir
- ML:
  - homojen
- **Temel işlemler:**
  - Head (ilk eleman)
  - Tail (geri kalan)

Union

- **Union Veri Tipi**
- **Tanım:**
- Aynı bellek alanında farklı türleri saklayabilen yapı
- 📌 **Free Union (C / C++)**
  - Tip kontrolü yok ❌
  - Hatalara açık
  - **Problem:**
    - Yanlış türde okuma → anlamsız veri
- 📌 **Discriminated Union (Ada, ML, F#)**
- Tag (etiket) içerir
- Tip kontrolü var ✔️
- **Avantaj:**
  - Güvenli
  - Hatalar önlenir



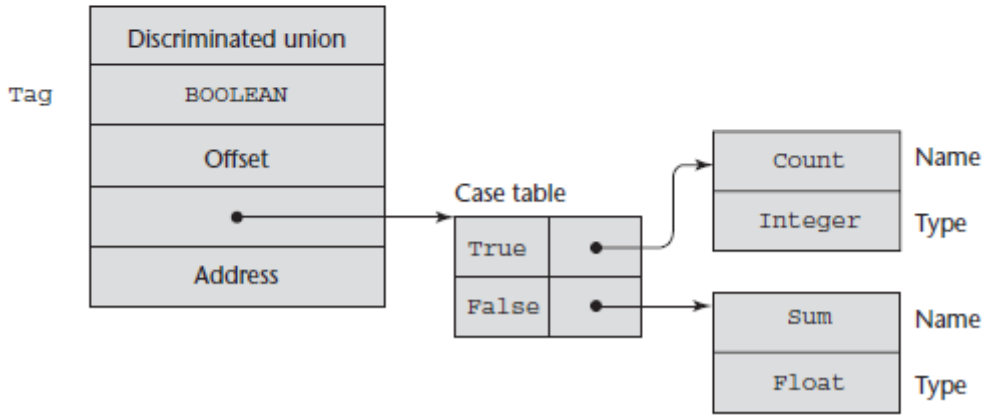
Union

- Union'lar, her olası varyant için aynı bellek adresi kullanılarak gerçekleştirilir. En büyük varyant için yeterli miktarda bellek ayrılır. Ayrımlı (discriminated) union'larda etiket (tag), varyant ile birlikte kayıt benzeri bir yapı içinde saklanır.
- Derleme zamanında, her varyantın tam tanımının saklanması gerekir. Bu, tanımlayıcıdaki (descriptor) etiket girdisiyle ilişkilendirilmiş bir case tablosu kullanılarak yapılabilir. Case tablosu, her varyant için bir giriş içerir ve bu giriş ilgili varyantın tanımlayıcısına işaret eder.
- Bu düzeni göstermek için aşağıdaki Ada örneğini ele alalım.

Derleme Zamanında

```
type Node (Tag : Boolean) is
  record
  case Tag is
      when True => Count : Integer;
      when False => Sum : Float;
  end case;
 end record;
```

The descriptor for this type could have the form shown in Figure 6.9.



Pointer

- Pointer → Değeri **bellek adresi** olan veri tipi
- Özel değer: nil → geçersiz/boş adres
- **Temel Kullanım Amaçları:**
  - **Dolaylı adresleme (indirect addressing)**
    - Assembly seviyesine yakın kontrol sağlar
  - **Dinamik bellek yönetimi**
    - Heap üzerinde oluşturulan verilere erişim

- **Heap-Dynamic Değişkenler:**
  - Heap'te runtime'da oluşturulur
  - Genellikle **isimsiz (anonymous)**
  - Sadece pointer/reference ile erişilir
- **Tür Ayrımı:**
  - Pointer → **reference type**
  - Normal değişken → **value type**
- **Dil Tasarım Yaklaşımları:**
  - C/C++ → esnek ama tehlikeli
  - Java → pointer yok, sadece **reference**
  - Ada → daha kontrollü yapı
    - Bellek yönetimi kontrollüdür.
    - Pointer'a doğrudan integer değer atamak genellikle yasaktır.
    - Pointer sadece doğru türdeki objeleri gösterebilir.

Pointer

- **Assignment (Atama)**
  - Pointer’a adres atanır
  - C Kaynak:
    - & operatörü (adres alma)
    - malloc, new
- **Dereferencing (Çözme)**
  - Pointer’ın gösterdiği değere erişim
  - **Örnek:**
    - ptr = 7080
    - *ptr = 206
    - j = *ptr → 206
- **C/Struct ile Kullanım:**
  - (*p).field
  - p->field (syntactic sugar)
- **Bellek Yönetimi:**
  - Allocation:
    - malloc, new
  - Deallocation:
    - free, delete

Pointer

- Dangling Pointer Oluşum Senaryosu:

  p1 → heap object

  p2 = p1 (aliasing)

  free(p1)

  p2 → geçersiz adres ❌

```
int * arrayPtr1;
int * arrayPtr2 = new int[100];
arrayPtr1 = arrayPtr2;
delete [] arrayPtr2;
// Now, arrayPtr1 is dangling, because the heap storage
// to which it was pointing has been deallocated.
```

Kayıp Heap-Dinamik Değişkenler

- **Tanım**
  - Kayıp heap-dinamik değişken, **heap üzerinde ayrılmış fakat artık programa erişilemeyen değişkendir**.
  - Bu değişkenler genellikle "**garbage (çöp)**" olarak adlandırılır:
    - Orijinal amaçları için kullanılamazlar.
    - Programda yeni bir amaçla yeniden kullanılamazlar.

- **Oluşma Süreci**
  - Pointer p1, yeni bir heap-dinamik değişkene işaret eder.
  - Sonra p1, başka bir yeni heap-dinamik değişkene yönlendirilir.
  - 🔹 **Sonuç:** İlk değişken artık ulaşılamaz, kaybolur.

```
int* p1 = new int(10);
p1 = new int(20);
```

Kayıp Değişkenler

- Bellek sızıntısı, **hem otomatik (garbage-collected) hem de manuel deallocasyon yapılan dillerde** sorun yaratır.
- Kayıp değişkenler, **programın performansını ve güvenilirliğini** etkiler.

- **Çözüm Yöntemleri**
  - **Garbage collection:** Dil tarafından kullanılmayan heap alanlarının otomatik temizlenmesi.
  - **Explicit deallocation:** delete veya free gibi işlemlerle programmer tarafından bellek temizliği.
  - **Pointer yönetimi:** Dangling pointer ve kayıp değişkenler için dikkatli kod yazımı.

Referans Tipleri

- Referans tip değişkenler, pointer'a benzer, ama önemli fark:
  - Pointer: Bellek adresine referans verir.
  - Referans: Bellekteki bir objeye veya değere referans verir.
- Matematiksel işlem yapmak adresler üzerinde mantıklı, referanslar üzerinde mantıklı değil.

```
int result = 0;
int &ref_result = result;  // ref_result, result'un alias'ı
ref_result = 100;          // result da 100 olur
```

- Referans tipleri formal parametre olarak iki yönlü iletişim sağlar.
- C++: Referans sabittir, başka bir değişkene atanamaz. İmplicit dereference vardır.
- Java: Tüm class instance'ları referans değişkenler aracılığıyla kullanılır. Referanslar değiştirilebilir ve implicit deallocation vardır.
- C#: Java referansları + C++ pointer'ları. Pointers kullanımı unsafe ile sınırlı.
- Diğer: Smalltalk, Python, Ruby, Lua – tüm değişkenler referans tiptir, implicit dereference yapılır.

Heap Yönetimi

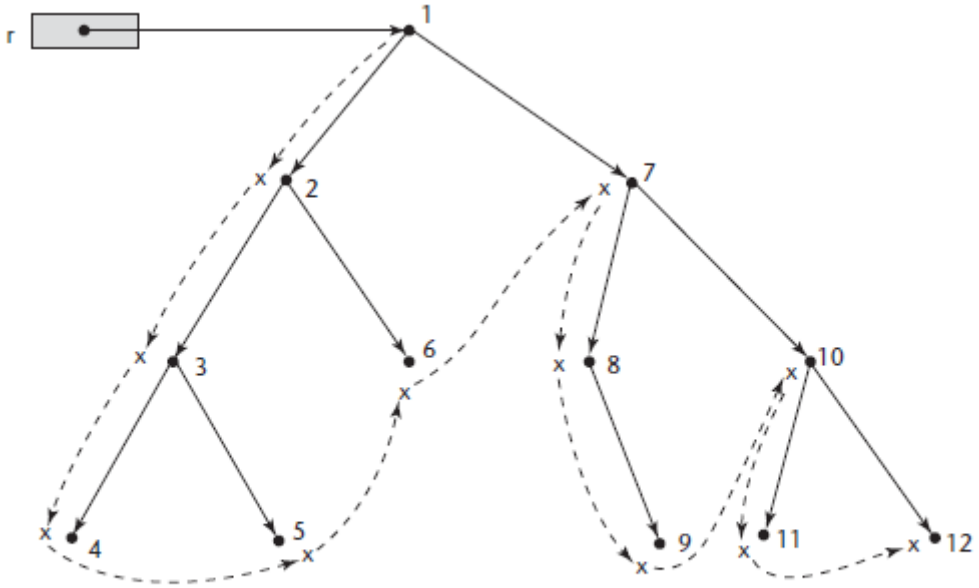
- Heap yönetimi pointer/referans ile dinamik bellek yönetimi sağlar.
- Dangling pointer çözümleri:
  - Tombstones: Her heap değişkeni özel hücre içerir; deallocated hücrede nil işareti.
  - Locks-and-Keys: Pointer = (key, address), heap variable lock değeri ile karşılaştırılır.
  - En güvenli çözüm: Implicit deallocation – programcı manuel dispose yapmaz (Java, C#).
- Heap yönetimi:
  - Tek boyutlu hücreler: LISP tarzı linked list kullanımı.
  - Variable boyutlu hücreler: Mark-sweep veya reference counting ile yönetim.
- Garbage Collection teknikleri:
  - Reference counters (incremental)
  - Mark-sweep (lazy, tüm heap taraması)
- İyileştirmeler: Incremental mark-sweep, parça parça mark-sweep, pointer rotation/slide.

Heap Yönetimi

- Referans Sayacı (Reference Counter) Yöntemi
  - Her hücrede, o hücreye işaret eden göstericilerin (pointer) sayısını tutan bir sayaç bulunur.
  - Bir gösterici hücreden ayrıldığında sayaç 1 azaltılır.
  - Sayaç 0'a ulaştığında, hücreye artık işaret eden gösterici yoktur → hücre "çöp" olarak işaretlenir ve kullanılabilir hafıza listesine eklenir.
- Artımlı (incremental) bir yöntemdir ve uygulamanın çalışmasını önemli ölçüde geciktirmez.
- Hücreler küçükse, sayaçlar için gereken ekstra hafıza büyük bir yük oluşturur.
- Her pointer değişiminde: Eski hücre sayacı azaltılır, Yeni hücre sayacı arttırılır.
- LISP gibi pointer değişikliklerinin sık olduğu dillerde performans düşebilir.
- Çözüm: "Deferred reference counting" yöntemi ile bazı pointerlar için sayaç tutulmayabilir.
- Dairesel Referans Problemi:
- Döngüsel bağlı hücrelerde her hücre sayacı ≥ 1 olduğu için, bu hücreler çöp olarak toplanamaz.
- Friedman ve Wise (1979) bu probleme çözüm önermişti

Heap Yönetimi

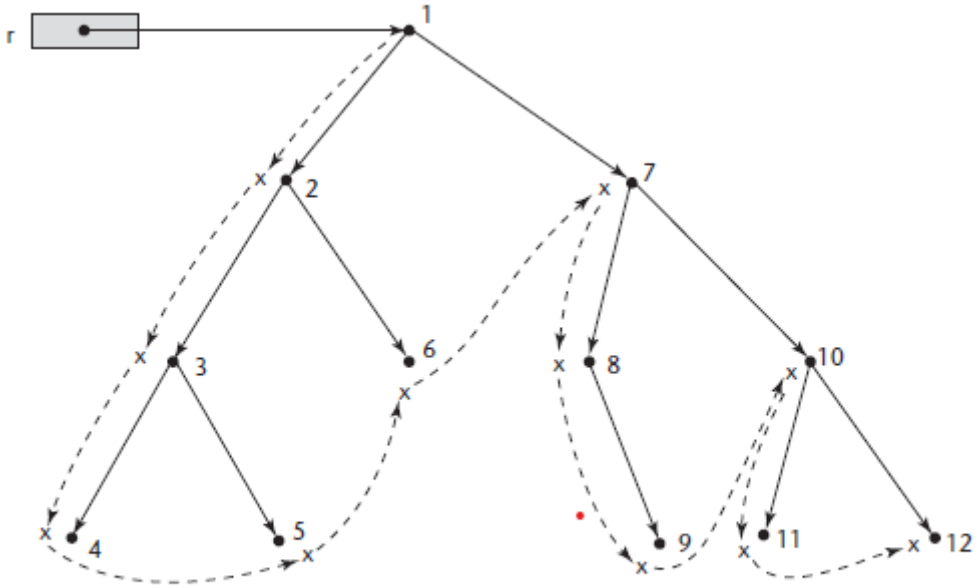
- **Mark-Sweep Yöntemi**
- **Tanım:**
  Geleneksel çöp toplama yöntemi (garbage collection) üç aşamalıdır:
- **İşaretleme Öncesi:**
  - Tüm heap hücreleri başlangıçta “çöp” olarak işaretlenir.
- **Marking (İşaretleme) Aşaması:**
  - Programdaki tüm pointerlar izlenir.
  - Erişilebilen hücreler “çöp değil” olarak işaretlenir.
- **Sweep (Temizleme) Aşaması:**
  - İşaretlenmeyen tüm hücreler kullanılabilir hafıza listesine eklenir.
- **Hücre Yapısı:**
  - Bilgi alanı
  - Marker alanı (işaretleme için)
  - llink ve rlink pointerları
  - Amaç: Hücreler ile yönlendirilmiş çizeler oluşturmak ve ağaçları tarayarak kullanılan hücreleri işaretlemek.



Dashed lines show the order of node_marking

Heap Yöntemi

- İşaretleme Algoritması (Marking Algorithm) Örneği)
- Hedef: Heap üzerindeki tüm kullanılabilir hücreleri belirlemek.
- Yöntem:
  - Spanning tree (kaplayıcı ağaçlar) kullanılarak graf üzerindeki tüm hücreler taranır.
  - Tarama sırasında bulunan her hücre “kullanılıyor” olarak işaretlenir.
  - Algoritma rekürsif olarak çalışır (her düğümden iki pointer ile diğer düğümlere geçiş).
- Avantaj:
  - Heap üzerindeki kullanılmayan tüm hücreleri güvenle toplar.
  - Döngüsel referans problemi yoktur (reference counting’in tersine).



Dashed lines show the order of node_marking

Tip Denetimi

- Operatörler → fonksiyonlar (subprogramlar) da operatör gibi düşünülür
- Atama (=) → iki operandlı (binary) operatör
  - Sol: hedef değişken
  - Sağ: ifade (expression)
- Amaç: Operatörlerin operandlarının **uyumlu tipte** olup olmadığını kontrol etmek
- Compatible type (uyumlu tip):
  - Doğrudan geçerli tip
  - veya coercion (örtük dönüşüm) ile geçerli hale getirilen tip
- **Coercion (Örtük Tip Dönüşümü):**
  - Derleyici/interpreter tarafından otomatik yapılır
  - Örnek:
    - int + float → float
    - int değeri otomatik float'a çevrilir
- Operatörün yanlış tipte operand ile kullanılması
  - Örnek:float bekleyen fonksiyona int gönderilmesi (eski C'de kontrol yoktu)

Statik vs Dinamik Tip Denetimi

- Static Type Checking (Derleme Zamanı):
  - Tüm tip bağlamaları statik ise hatalar compile time'da yakalanır.
  - Avantaj:
    - Hatalar erken tespit edilir
    - Daha güvenli ve okunabilir kod
  - Dezavantaj:
    - Esneklik azalır
- Dinamik tip bağlama varsa runtime'da kontrol gerekir.
- Örnek diller:JavaScript, PHP
- Avantaj:
  - Esneklik yüksek
- Dezavantaj:
  - Hatalar geç fark edilir (daha maliyetli)

Statik ve Dinamik Tip Denetimi

- Aynı bellek alanı farklı tipler tutabiliyorsa:
  - C/C++ → union
  - Ada → variant record
  - ML, Haskell, F# → discriminated union
- Bu durumda:Tip kontrolü runtime'da yapılmak zorunda
- Sistem mevcut değerin tipini takip eder

- **Önemli Nokta:**
- Dil statik tipli olsa bile:
  - **Tüm hatalar compile time'da yakalanamaz**.
  - Bazıları runtime'da ortaya çıkar.

Güçlü Tip Sistemi

- Bir programlama dili **strongly typed** ise:
- **Tüm type hataları mutlaka tespit edilir**
- Bu tespit:
  - Derleme zamanında (compile-time)
  - veya çalışma zamanında (run-time)
- Tüm operandların tipi belirlenebilir olmalıdır.

- Avantajları:
  - Hatalı değişken kullanımı engellenir.
  - Tip uyumsuzlukları yakalanır
  - Daha güvenli ve güvenilir yazılım
- Özel Durum:
  - Çok Tipli Bellek (Union vb.) Aynı değişken farklı tip değerler tutabiliyorsa tip kontrolü runtime'da yapılır
  - Sistem mevcut değerin tipini takip eder.
- Ada: Neredeyse strongly typed Unchecked_Conversion ile tip kontrolü bypass edilebilir.
- ML, F#, Scala: Tam anlamıyla strongly typed

Güçlü Tipler ve Coercion Etkisi

- Strong Typing'i Zayıflatan Faktör: Coercion
- Otomatik tip dönüşümü → bazı hataları gizler.
- Örnek:int + float → float
- Yanlış değişken kullanımı fark edilmeyebilir
  - Örnek Senaryo:Doğru: a + b (int + int)
  - Hata: a + d (int + float)
  - Sonuç:a otomatik float'a çevrilir
  - Hata yakalanmaz
- C, C++:Strongly typed değil.
  - Çok fazla coercion → düşük güvenilirlik.
  - union → type checking yok
- Java, Scala, C#:Strong typing var
  - Ama sınırlı coercion → bazı hatalar kaçabilir.
- ML, F#:Coercion yok → en güçlü hata tespiti.

Tip Uyumluluğu ve Tip Eşitliği

- Operatörlerin hangi tiplerle çalışabileceğini belirler
- Uyum, iki şekilde sağlanır:
  - **Doğrudan uygunluk**
  - **Coercion (örtük dönüşüm)** ile uygunluk
- **Örnek:**
  - int + float → float (int otomatik dönüştürülür)
- **Type Error (Tip Hatası):**
  - Operatör + uyumsuz tip kombinasyonu
- **Type Equivalence (Tip Eşdeğerliği)**
  - Daha katı bir kavramdır
- **Coercion olmadan** iki tipin değiştirilebilir olması gerekir
  - 👉 Özet:
  - Compatibility = Esnek (coercion var)
  - Equivalence = Katı (coercion yok)

Tip Uyumluluğu ve Tip Eşitliği

- Name Type Equivalence (İsim Bazlı)
- İki değişken aynı tiptedir eğer aynı isimle tanımlanmışsa
  - ✓ Avantaj:Basit ve hızlı kontrol
  - ❌ Dezavantaj:Çok katı (aynı yapıda olsa bile farklı sayılır.)

```
type IndexType is 1..100;
count : Integer;
index : IndexType;
```

- Structure equivalence:
  - Esnek ama anlamsal hatalara açık
- Name equivalence:
  - Güvenli ama katı

- Structure Type Equivalence (Yapısal)
- Tipler aynıysa değil, yapıları aynıysa eşdeğer sayılır
  - ✓ Avantaj:Daha esnek
  - ❌ Dezavantaj:Karşılaştırma zor (özellikle recursive yapılarda)
- **Problemli durumlar:**
  - Alan isimleri farklı ama yapı aynı → eşdeğer mi?
  - Array indeksleri farklı → eşdeğer mi?
  - Enum değerleri farklı yazılmış → eşdeğer mi?

Veri Tipleri ve Tip Denetimi - Sorular

- “Pointers ile ilgili iki yaygın problem nedir?”
- “Uyumlu tür nedir?”
- Güçlü tür denetiminin en önemli özelliği nedir?
- Güçlü tür denetimini C/C++ sağlamadığı bir örnek ile gösteriniz?

```c
#include <stdio.h>

int main() {
    int x = 10;
    float y = 5.5;

    int *p = &x;
    float *q = &y;

    // Derleyici uyarı verebilir ama çoğu C derleyicisi çalıştırır
    p = (int*) q;  // float* → int* dönüştürmesi (cast ile)
    printf("%d\n", *p);  // Yanlış bellek okuması olabilir

    return 0;
}
```